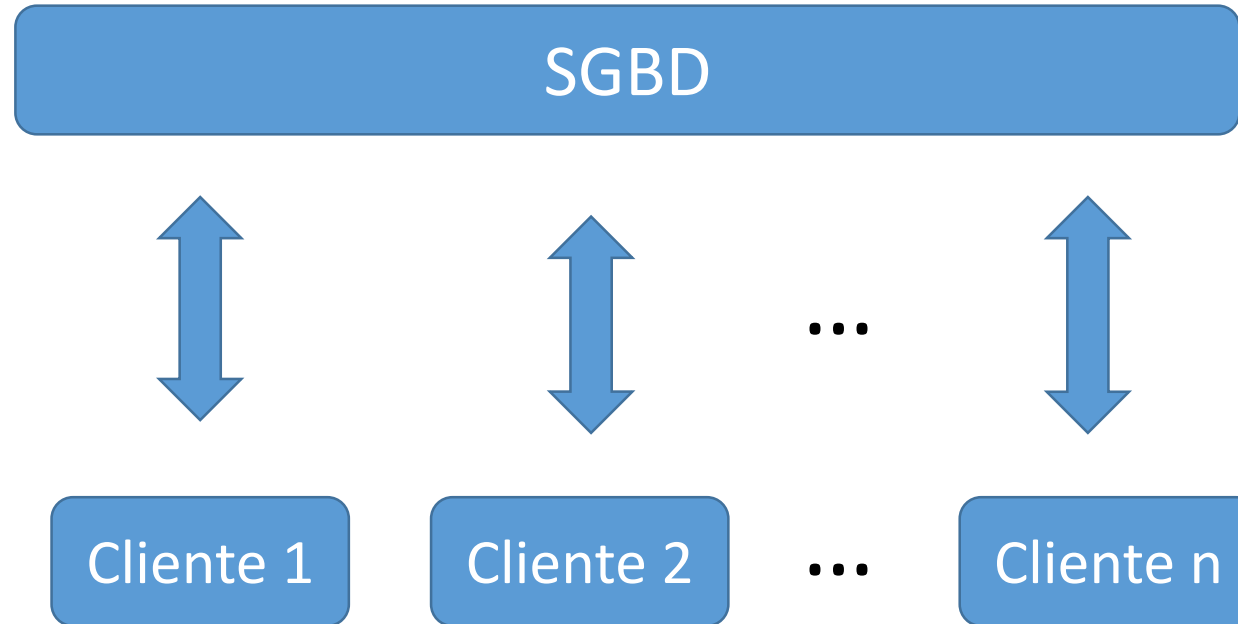
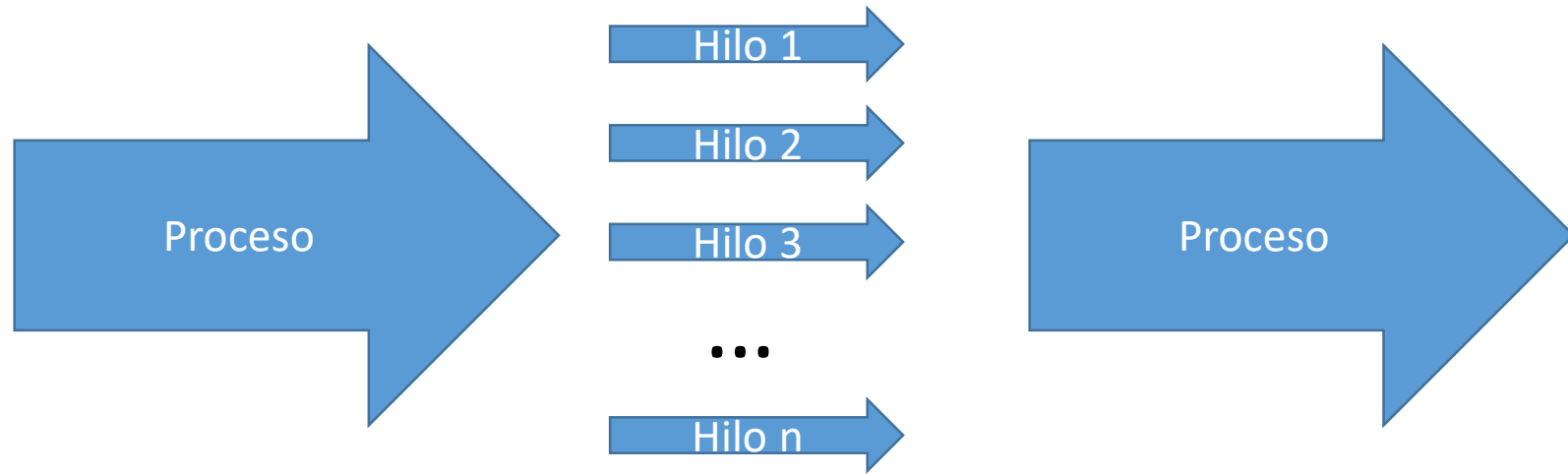


Transacciones. Concurrency

Concurrencia

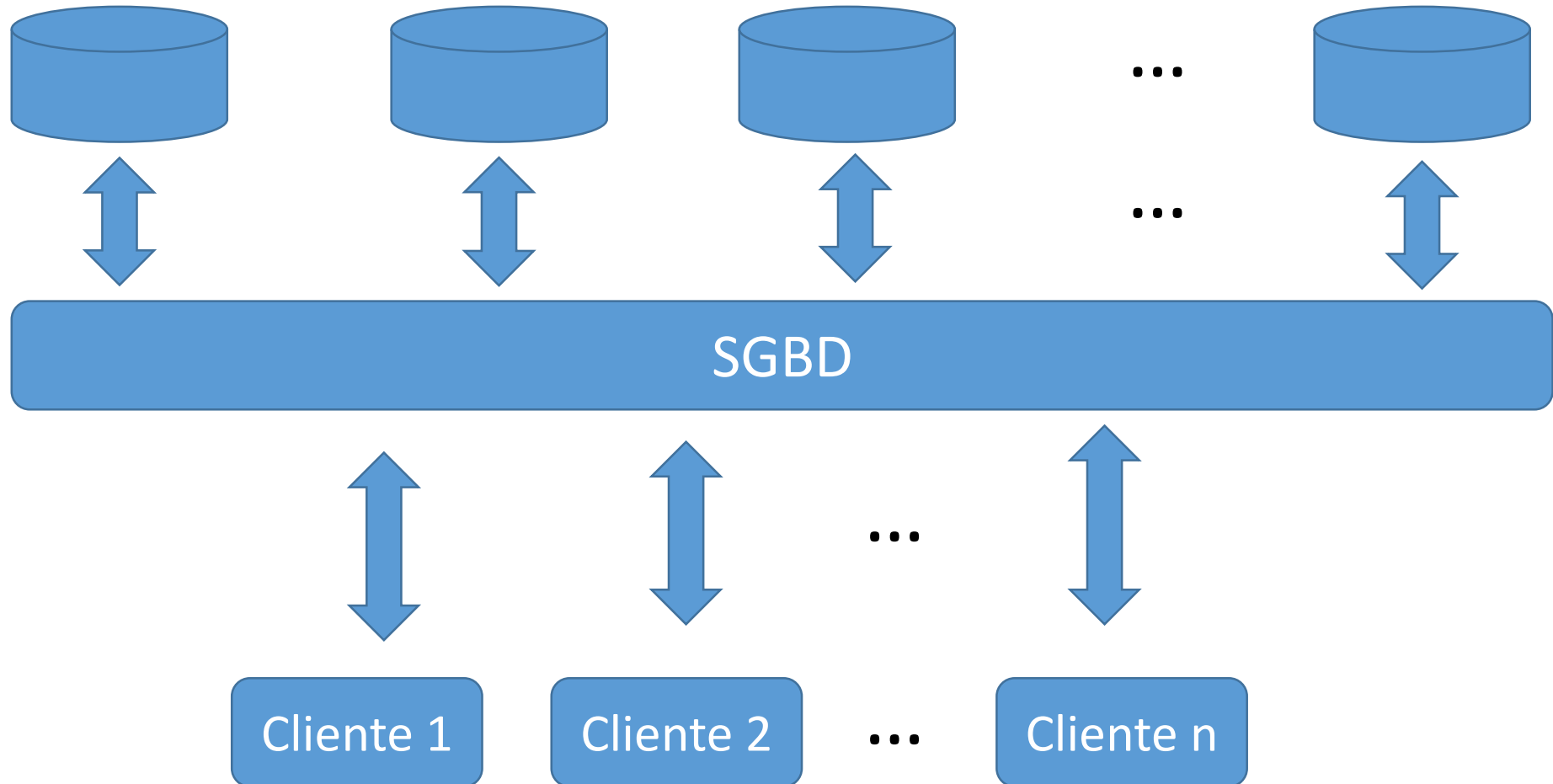


Concurrencia



Los hilos Comparten datos

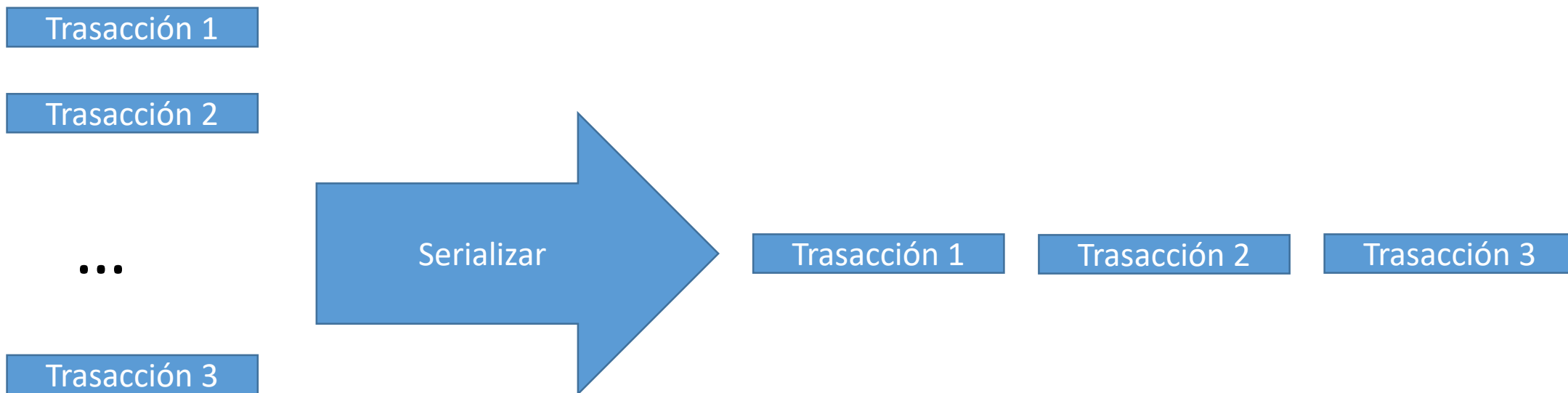
Concurrencia



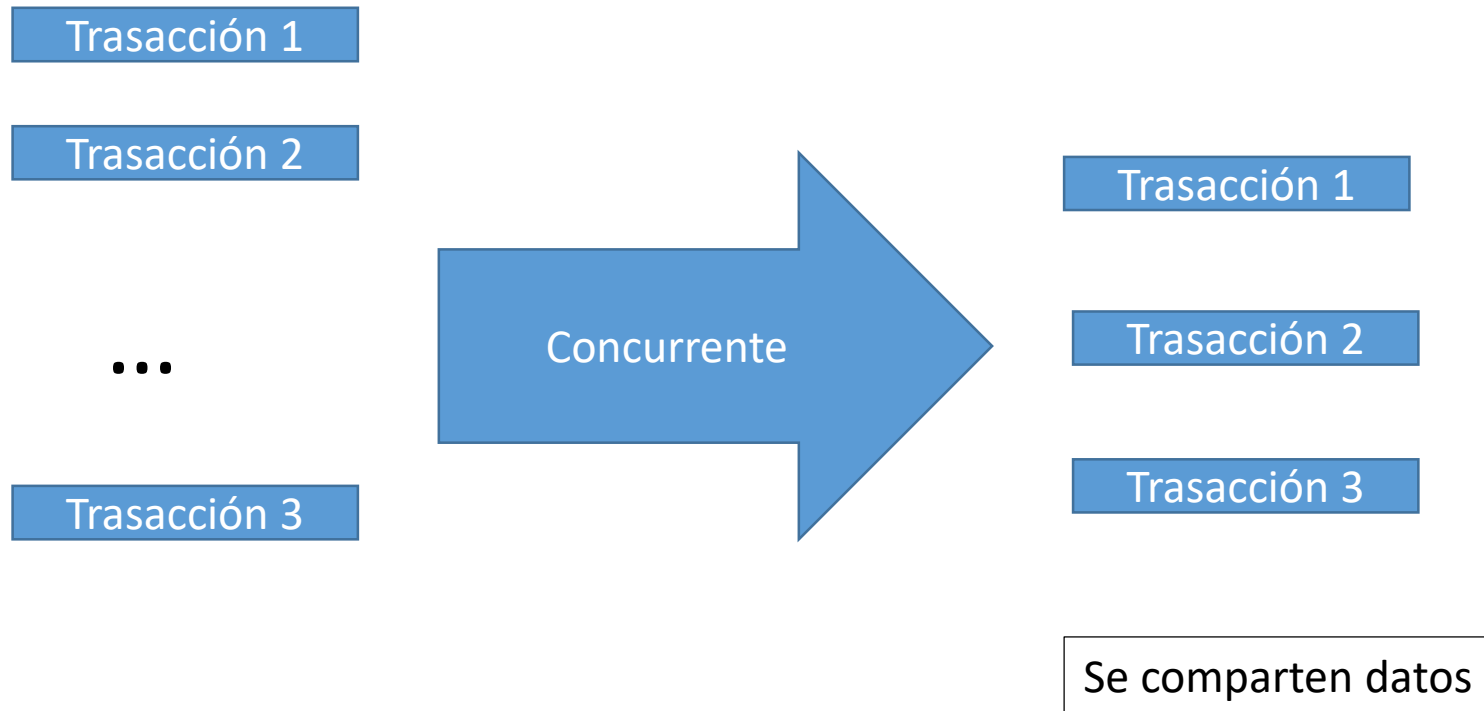
Transacción

Secuencia ordenada de instrucciones atómicas

Transacción



Transacción



Concurrencia solapada - interleaved concurrency

- Se tiene un solo procesador
- El procesador ejecuta una sola instrucción (de una transacción) a la vez
- El scheduler puede suspender una transacción y continuar con otra

Concurrencia solapada - interleaved concurrency

- Ninguna Transacción frena a otra
- Ejemplo: El join es muy costoso

Item

- El valor de un atributo en una fila determinada de una tabla.
- Una fila de una tabla.
- Un bloque del disco.
- Una tabla.

Tamaño del ítem -> Granularidad

Propiedades ACID

- Deseables para un SGBD
 - **A**tomicidad
 - **C**onsistencia
 - **A**islamiento
 - **D**urabilidad

Propiedades ACID

I. BEGIN TRANSACTION

II.

- **COMMIT**
- **ABORT(ROLLBACK)**

Anomalías - Lectura Sucia (Dirty Read)

- T1 y T2 don transacciones
- T1 modifica X
- T2 lee X
- T1 Aborta
 - El valor de X leído por T2 es falso

Tipo: Write Read

Temporary update

Read uncommitted data

Anomalías - Lectura Sucia (Dirty Read)

T1	T2	Stock (inicial=1000)	Leido
St1=Read (Stock)			1000
Write(Stock, st1-300)		700	
	St2=Read(Stock)		700
Aborta		1000	

Anomalías - Actualización Perdida (lost update)

- T1 y T2 son transacciones
- T1 lee X
- T2 modifica X
 - T1 modifica X (se pierde la modificación de T2)
 - T1 vuelve a leer X (X leído es distinto del anterior)

Tipo: Read Write

Lectura no repetible (unrepeatable read)

Anomalías - Actualización Perdida (lost update)

T1	T2	Stock (inicial=1000)	Leido
St1=Read (Stock)			1000
	St2=Read(Stock)		1000
Write(Stock, st1-300)		700	
	Write(Stock, st1+300)	1300	

Anomalías - Escritura sucia (dirty write))

- T1 y T2 son transacciones
- T1 lee X
- T1 escribe X
- T2 modifica X
- T1 aborta

Tipo: Write Write

Escritura sucia (overwriteuncommitted)

Anomalías - Escritura sucia (dirty write))

T1	T2	Stock (inicial=1000)	Leido
St1=Read (Stock)			1000
Write(Stock, st1-300)		700	
	St2=Read(Stock)		700
	Write(Stock, st1-800)	-100	
	commit		
Aborta		200	

Anomalías - Fantasma (phantom))

- T1 y T2 son transacciones
- T1 lee X (tabla)
- T2 modifica X
- T1 las condiciones pueden haber cambiado

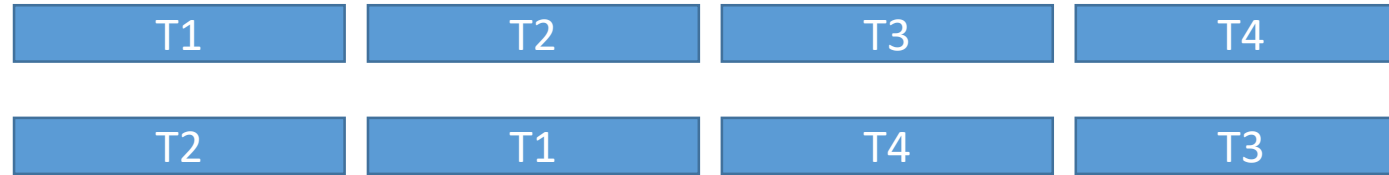
Anomalías - Fantasma (phantom)) - Ejemplo

Se envía una promo a todos los clientes de cierta categoría
Otra transacción inserta un nuevo cliente
La primera todavía no termino
Se perdió ese ultimo cliente agregado

No se resuelve serializando

Seriabilidad

Serie



Solapado



- Se respeta el orden de cada Transaccion
- Se separa en intruccines de Read y Write
- Es serializable cuando es equivalente a al menos una de la serializaciones
- Si son serialiazables -> están aisladas

Seriabilidad - Conflictos

Los conflictos se dan con los Write sobre un mismo item

$R_i(X) \ W_j(X)$:Una transacción escribe un ítem que otra leyó.

$W_i(X) \ R_j(X)$:Una transacción lee un ítem que otra escribió.

$W_i(X) \ W_j(X)$:Dos transacciones escriben un mismo ítem.

I y j sin transacciones X es un elemento

Seriabilidad - Conflictos

Dado este solapamiento entre T1y T2

RT1 (A);WT1 (A);RT2 (A);WT2 (A);RT1 (B);WT1 (B);RT2 (B);WT2 (B);

Conflictos A

WT1 (A);RT2 (A)

RT1 (A)WT2 (A)

WT1 (A)WT2 (A)

Conflictos B

WT1 (B);RT2 (B)

RT1 (B)WT2 (B)

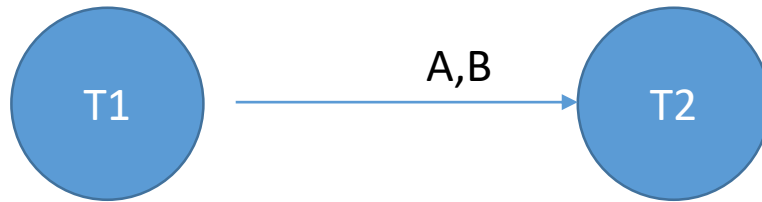
WT1 (B)WT2 (B)

RT1 (B);WT1 (B) puede ir antes de RT2 (A);WT2 (A)

RT1 (A);WT1 (A);RT1 (B);WT1 (B);RT2 (A);WT2 (A);RT2 (B);WT2 (B)

T1;T2 Com es equivalente a esta en serie -> es serializable

Seriabilidad - Grafo de precedencias

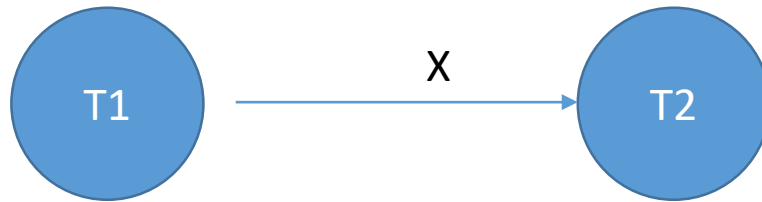


Vertices: Transacciones
Aristas: conflictos

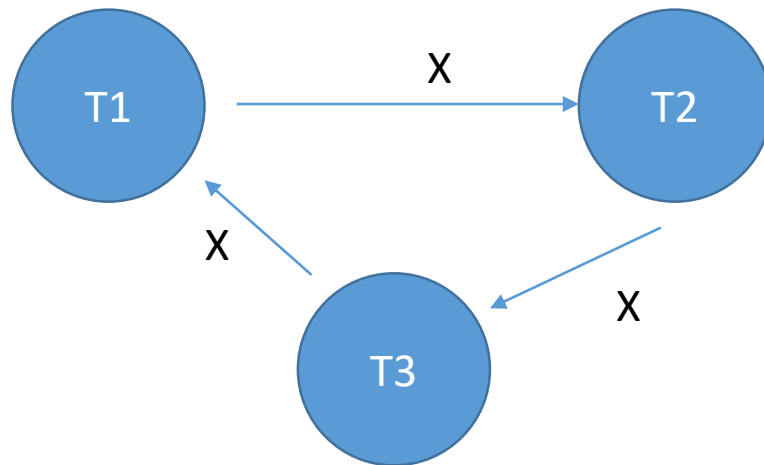
T1 debe preceder a T2 → No se pueden invertir

Si no tiene ciclos es serializable

Seriabilidad - Grafo de precedencias



Es Serializable



No es Serializable

Anomalías - Control de concurrencia

- Locks
- Basado en TimeStamps
- Isolation Snapshot

Anomalías - Control de concurrencia

- Criterio pesimista
- Criterio Optimista

Anomalías - Control de concurrencia

- Locks (pesimista)
- Basado en TimeStamps (optimista)
- Isolation Snapshot (optimista)

Anomalías - Control de concurrencia

- nivel de lockeo (lo maneja el SGBD)
- Fila
- tabla

Anomalías - Control de concurrencia

exclusión mutua (mutex) ->Sección Crítica

Acquire lock (lockear)

Release unlock (deslockear)

Anomalías - Control de concurrencia

transacción con lock no se interrumpe

Anomalías - Tipos de Lock

Escritura	Exclusivo	solo 1
Lectura	Compartido	1 o mas

Anomalías - lockeo en 2 fases

Lock 1

Lock 2

...

Lock n

...

Unlock n

..

Unlock 2

Unlock 1

Anomalías - lockeo Problemas

Deadlock
Inanición

Anomalías - DeadLock

- Prevenirlo
 - Prioridades de recursos
- Detectarlo
 - TimeOut

Inanición

Encolar pedidos

Anomalías - IndexLock

Se Lockea también el árbol B

Lockeo de predicados

- Lockea con condición

Anomalía fantasma

- Lock de predicados
- Lockeo de tabla

TimeStamp

- FechaHora actual
- Secuencia

TimeStamp

- Se toma un orden
- Si falla ->Aborta

TimeStamp

- Para cada ítem ultima lectura y ultima escritura

Por cada read si una T posterior(de acuerdo al orden preestablecido) la escribió → aborta

Pro cada write si una T posrerior(de acuerdo al orden preestablecido) la escribió o leyó → aborta

Snapshot Isolation

- Cada transacción ve su snapshot
- Al final decide si comitea o aborta (si hay conflicto con otra transacción)
- Mucha memoria